

Stitch-Local Crossing Semantics and an Algebraic Pipeline for Finite Knitted Closures

April 6, 2026

Purpose

This note turns the current memo into a mathematical work plan. The main goal is to define a stitch-local representation that is strong enough to support:

1. a constructive proof that a finished finite knitted object closes to a single component,
2. an algorithm that extracts a knot diagram from the stitch program, and
3. a local update rule for a computable invariant, preferably via Fox colorings or Alexander-type matrices.

The key conclusion is that the most efficient proof route is:

stitch program $\rightarrow$ yarn graph with boundary data $\rightarrow$ yarn crossing presentation $\rightarrow$ reduced coloring/Alexander m

Among the invariant options, the coloring-matrix/Alexander route is the one that naturally supports local row and column operations. Jones polynomials can still be computed after diagram extraction, but they are not the right primary proof engine for a stitch-by-stitch update theorem.

1. What Must Be Proved

There are really two different theorems.

Theorem family A: single closed component. This is the statement that after the final pass through the last bind-off loop and the identification with the cast-on end, the construction yields exactly one closed curve. This should be proved from the combinatorics of the yarn path, not from a determinant.

Theorem family B: knot invariant extraction. Once the closed curve is represented as a knot diagram, one can compute a determinant, Fox coloring matrix, Alexander polynomial, or Jones polynomial. These certify nontriviality or identify knot type, but they do not by themselves replace the single-component proof.

Remark 1. *The determinant is excellent for proving that the closure is nontrivial, or for distinguishing examples. It is not, by itself, the right certificate that the finished object is a knot rather than a multi-component link.*

2. Recommended Representation

2.1 Three levels

The existing project already has the first two layers.

1. **Stitch graph.** One vertex per stitch event, ordered left-to-right within rows and vertically by dependence.
2. **Yarn graph.** A directed multigraph recording the path of the working yarn through these stitch events, with explicit “up” and “down” edges.
3. **Yarn crossing presentation (YCP).** A combinatorial knot-diagram object that records arcs, crossings, and boundary endpoints.

The new work belongs almost entirely in level 3, but level 3 should be derived from level 2 by a local rule at each stitch.

2.2 The yarn crossing presentation

Definition 1 (Yarn crossing presentation). *A yarn crossing presentation is a tuple*

$$\text{YCP} = (\mathcal{A}, \mathcal{C}, \mathcal{F}, \prec, \partial_{\text{in}}, \partial_{\text{out}})$$

where:

1. $\mathcal{A}$ is the set of oriented arc segments between crossings,
2. $\mathcal{C}$ is the set of crossings,
3. each crossing $c \in \mathcal{C}$ has an ordered triple

$$c = (o(c); u_1(c), u_2(c), \varepsilon(c)),$$

where $o(c)$ is the over-arc, $u_1(c), u_2(c)$ are the under-arcs, and $\varepsilon(c) \in \{+1, -1\}$ is the crossing sign,

4. $\mathcal{F} \subseteq \mathcal{A}$ is the ordered frontier of arcs still exposed to future knitting operations,
5. $\prec$ is the left-to-right order on the current frontier,
6. $\partial_{\text{in}}, \partial_{\text{out}}$ are the initial cast-on and active working-yarn boundary endpoints.

The frontier $\mathcal{F}$ is the crucial object. It is what allows a stitch to be added locally. Each new stitch consumes a constant number of frontier arcs and produces a constant number of new frontier arcs.

2.3 Why the yarn graph is not quite enough

The yarn graph captures adjacency of yarn segments around stitch events, but it does not by itself specify which local strand is over and which is under. Therefore, for each stitch vertex v , one must add a *local crossing signature*

$$S(v) = (\text{interface arcs}, \text{internal arcs}, \text{crossings}, \text{frontier update}).$$

This signature is the missing data needed to derive a knot diagram.

3. Local Stitch Signatures

3.1 General form

For each stitch type σ , define a constant-size gadget G_σ with:

1. consumed interface arcs I_σ^- ,
2. produced interface arcs I_σ^+ ,
3. internal arcs J_σ ,
4. local crossings C_σ ,
5. a gluing rule identifying I_σ^- with specific entries of the current frontier $\mathcal{F}$,
6. an ordered replacement rule that inserts I_σ^+ back into the frontier.

This lets every stitch act as a rewrite step

$$(\text{YCP}_t, \mathcal{F}_t) \xrightarrow{\sigma_t} (\text{YCP}_{t+1}, \mathcal{F}_{t+1}).$$

3.2 Baseline stitch classes

The exact signs depend on the orientation convention in the final diagrams. The table below therefore records the *combinatorial* update type first and treats signs as a convention choice.

Stitch type	Consumes	Produces	Local crossing data
First cast-on	0 frontier loops	1 live loop, plus initial tail/work strand boundary data	In the simplest untwisted model, no intrinsic crossing; initializes ∂_{in} and $\mathcal{F}$.
Generic cast-on	0 live loops	1 new live loop	Usually no intrinsic crossing in the abstract model; appends one new frontier arc and one horizontal yarn adjacency.
Wrap-turn	1 current live loop	1 wrapped live loop	Adds one wrap arc around the held loop; one local signed crossing is the natural minimal model.
Knit	1 old loop	1 new loop	One local crossing between the working strand and the consumed loop; sign convention opposite to purl.
Purl	1 old loop	1 new loop	Same incidence pattern as knit, but opposite crossing sign.
Bind-off	1 old loop, plus interaction with previous bind-off loop	usually 1 continuing terminal loop until the last step	Contains the knit-like crossing plus the pass-through that links successive bind-off loops; this is the first place where frontier size can decrease.
Final bind-off pull-through	final live loop	0 live loops	Closes the yarn path by identifying the working end with the remaining terminal loop; if embedded in a boundary collar, it can be chosen crossing-free.

Remark 2. *For the first paper, it is completely reasonable to adopt a minimal crossing semantics: cast-on is crossing-free, knit and purl contribute one signed crossing each, wrap-turn contributes one signed crossing, ordinary bind-off contributes a bounded constant number of crossings, and the final pull-through is crossing-free in a collar neighborhood. This gives a clean theorem surface.*

3.3 The “one missing arc” principle

Your current observation can be formalized as follows.

Claim 1. *For a generic one-loop stitch, the yarn graph already determines all but one arc in the corresponding local knot-diagram gadget. The remaining arc can be canonically assigned to the stitch vertex itself.*

This is important because it means the crossing presentation need not duplicate the entire yarn graph. Instead, for each stitch vertex v , store:

1. the incident yarn-graph edges that already specify inherited arcs,
2. one distinguished residual arc $\rho(v)$,
3. the local crossing list in terms of those inherited arcs and $\rho(v)$.

That gives a compact representation:

$$v \mapsto (\rho(v), C(v), \text{frontier action of } v).$$

This is probably the most efficient bridge between the yarn graph and the crossing presentation.

3A. Current Stitch Library and Existing Connection Data

3A.1 What is already encoded in the project

The current codebase already records more structure than just stitch names. For the purposes of a knot-theoretic semantics, the useful fields are:

1. **Row/column placement.** The stitch graph stores row order and left-to-right order.
2. **Horizontal adjacency.** A horizontal edge records consecutive stitches produced in the same row.
3. **Vertical dependence.** A vertical edge records that a new stitch is worked through a previously live stitch.
4. **Consume versus keep behavior.** The stitch definition records whether a source loop is removed from the needle or remains live.
5. **Cursor advance.** The number of stitches consumed from the working frontier is encoded by `cursor_inc`.
6. **Production count.** The number of new stitch vertices is encoded by `add` and `extra`.
7. **Direction reversal.** A turn swaps left and right needle roles and changes the active frontier order.
8. **Topology markers.** Join and Möbius markers identify the first and last stitches in the cast-on frontier.

The fields that are *not* yet explicit enough for knot calculations are:

1. which local strand is over and which is under,
2. how many diagram arcs are associated to one stitch event,
3. which crossings are induced by a stitch,
4. how the active frontier of arcs changes after each stitch.

3A.2 Table of current stitch definitions

Table 1 documents the explicit stitch library in `stitches.txt` together with the special cable family handled in the parser.

Table 1: Current stitch definitions and the connection data they contribute.

Stitch	char	cursor_inc	keep	new sts	Connection data already encoded
‘k‘	k	1	no	1	One new stitch; horizontal adjacency to previous stitch in row; one consumed predecessor loop via a vertical edge.
‘p‘	p	1	no	1	Same graph-level data as ‘k‘; the knot-theoretic difference must therefore be carried by crossing sign/orientation, not by the current graph edges alone.
‘co‘	k	1	yes	1 via ‘extra‘	Cast-on creates a new live stitch without consuming an older loop; horizontal adjacency only. The first cast-on is a boundary initializer, later cast-ons append to the cast-on frontier.
‘yo‘	k	1	yes	1 via ‘extra‘	Like cast-on at graph level: adds a new live loop with horizontal adjacency but no consumed predecessor loop.
‘bo‘	k	1	no	0	Consumes one live stitch and creates no new stitch vertex. This should be interpreted as a frontier-reduction move in the crossing semantics.
‘kfb‘	k	1	no	2	One consumed predecessor loop produces two new stitch vertices. Horizontal adjacency links the two outputs; both are attached to the consumed loop.
‘kfb3‘	k	1	no	3	One consumed predecessor loop produces three new stitch vertices in one row segment.
‘kfb4‘	k	1	no	4	One consumed predecessor loop produces four new stitch vertices.
‘kfb5‘	k	1	no	5	One consumed predecessor loop produces five new stitch vertices.
‘kfb3-3‘	k	3	no	3	Consumes three predecessor loops and produces three new stitch vertices; this is already a many-to-many dependence gadget at graph level.
‘ssk‘	k	2	no	1	Consumes two predecessor loops and produces one new stitch; graph-level decrease stitch.
‘k2tog‘	k	2	no	1	Same graph-level arity as ‘ssk‘; the distinction will have to appear in local crossing order if the two are diagrammatically different.
‘p2tog‘	p	2	no	1	Purl-character decrease stitch with two consumed predecessors and one output.

Stitch	char	cursor_inc	keep	new sts	Connection data already encoded
‘k3tog‘	k	3	no	1	Three predecessor loops merged into one new stitch.
‘turn‘	k	0	yes	0	No new stitch vertex; swaps needle direction and reverses how future stitches encounter the frontier. This is the natural graph-level precursor to wrap-turn semantics.
‘join‘	k	0	yes	0	Topology marker for circular knitting; identifies the last cast-on stitch with the first by a horizontal connection.
‘mobius‘	k	0	yes	0	Topology marker for Möbius knitting; identifies the last cast-on stitch with the first under a half-twist convention.
‘cnf‘, ‘cnb‘ family	k	n	no	n	Implemented in the parser as cable moves: split n stitches into two halves, work one half before the other, and reconnect vertically/horizontally in that altered order. These are not yet explicit JSON stitch records.

Remark 3. *The table makes one structural point very clear: many stitch distinctions that matter diagrammatically are currently invisible at the graph-edge level. In particular, ‘k‘ versus ‘p‘, ‘ssk‘ versus ‘k2tog‘, and ordinary turn versus wrapped turn require extra crossing-order data.*

3A.3 Suggested extra fields for a mathematical stitch library

If the stitch file is extended for knot calculations, the minimal additional fields should be:

1. number of interface arcs consumed,
2. number of interface arcs produced,
3. ordered list of local crossings with signs,
4. residual arc identifier for any arc not already inherited from the yarn graph,
5. frontier replacement order,
6. boundary action for cast-on and final finishing moves.

4. Algebraic Extraction by Fox Colorings

4.1 Global matrix

Given a completed knot diagram with arc set $\mathcal{A} = \{a_1, \dots, a_m\}$ and crossings $\mathcal{C} = \{c_1, \dots, c_r\}$, define the integer coloring matrix $M \in \mathbb{Z}^{r \times m}$ by one row per crossing:

$$2x_{o(c)} - x_{u_1(c)} - x_{u_2(c)} = 0.$$

Equivalently, the row for crossing c has:

1. 2 in the column of the over-arc,
2. -1 in the columns of the two under-arcs,
3. 0 elsewhere.

For a knot diagram, deleting one row and one column yields a reduced coloring matrix M^{red} , and

$$\det(K) = \left| \det(M^{\text{red}}) \right|.$$

4.2 Why this is the right primary invariant engine

This representation is local in exactly the right way:

1. every new crossing contributes one sparse row,
2. every newly created arc contributes one column,
3. every identification of arcs is a column merge or column elimination,
4. every stitch therefore induces only constant-size matrix surgery.

That is the algebraic counterpart of stitch locality.

4.3 Frontier matrix update

Instead of rebuilding the full matrix after every stitch, keep a *frontier-reduced matrix* R_t whose columns correspond only to current frontier arcs plus a bounded set of recently introduced internal arcs not yet eliminated.

Suppose stitch σ adds a local block

$$B_\sigma = \begin{bmatrix} B_{\text{ff}} & B_{\text{fi}} \\ B_{\text{if}} & B_{\text{ii}} \end{bmatrix},$$

where the f -columns correspond to frontier arcs and the i -columns correspond to internal arcs introduced by the local gadget. After gluing consumed frontier arcs to the appropriate columns of R_t , eliminate the internal columns by Gaussian elimination or Schur complement. The updated state is:

$$R_{t+1} = \text{Reduce}(R_t \oplus B_\sigma).$$

In words, each stitch contributes a constant-size block and then a constant-size elimination. This is the cleanest formal version of “the stitch induces row operations on a coloring matrix.”

Remark 4. *To prove correctness, you do not need to claim that a stitch acts by only one elementary row operation. In general it will be a bounded sequence of row operations, column insertions, column identifications, and elimination of local internal variables.*

4.4 Better formal statement

The theorem to aim for is:

Theorem 1 (Local algebraic update theorem). *Let Σ be a finite library of stitch primitives, each equipped with a crossing signature G_σ . Then there exists a finite set of matrix update templates $\{U_\sigma : \sigma \in \Sigma\}$ such that for every valid stitch program $P = \sigma_1 \cdots \sigma_N$, the reduced coloring matrix of the final closure can be obtained by iteratively applying the updates U_{σ_i} to the frontier-reduced matrix state.*

This is the exact row-operation style theorem that appears to fit the project best.

5. Alexander and Jones Options

5.1 Alexander route

The Alexander matrix is also local enough to work stitch-by-stitch. One can derive it from the same crossing presentation by writing Wirtinger generators for arcs and Fox derivatives for crossings, then evaluating in the abelianization. This is more algebraically involved than the coloring matrix, but it is still linear and local.

Recommendation: prove the local update theorem first for the coloring matrix, then package the Alexander matrix as a stronger extension using the same crossing presentation.

5.2 Jones route

The Jones polynomial is less attractive as the primary proof tool because a skein relation branches recursively. It is excellent as a post-processing invariant once the final diagram is known, but it is not naturally described by bounded local matrix updates at each stitch.

Recommendation: do not build the main theorem around Jones updates. Use Jones only after the crossing presentation is complete, or for small examples.

6. Proof Strategy Split Into Two Pipelines

6.1 Pipeline A: prove the result is a knot

This should be proved from the yarn graph and frontier semantics.

Definition 2 (Single-strand state). *A knitting state is single-strand if the yarn graph contains exactly one path component with two boundary endpoints: the cast-on end and the active working end.*

Lemma 1 (Local preservation of single-strandness). *Each stitch in the permitted library transforms a single-strand state into a single-strand state.*

Theorem 2 (Closure theorem). *If a stitch program begins in the first cast-on state, each stitch preserves single-strandness, and the final finishing move identifies the active working end with the cast-on end through the terminal loop, then the resulting finished object is one closed component.*

This theorem proves “it is a knot” in the topological sense of a single closed curve.

6.2 Pipeline B: prove or compute nontriviality

Once closure is established, use the crossing presentation to compute an invariant:

1. determinant via reduced coloring matrix,
2. Alexander polynomial via Fox derivatives,
3. Jones polynomial as optional post-processing.

This theorem should look like:

Theorem 3 (Invariant extraction theorem). *For every valid closed stitch program in the primitive library, the corresponding crossing presentation can be transformed in polynomial time into a reduced coloring matrix and hence into $\det(K)$. The same crossing presentation also determines a Wirtinger presentation and an Alexander matrix.*

7. Concrete Stitch-by-Stitch Data To Record

For each primitive σ , I recommend that the formal library include the following exact fields:

1. **Consumption data:** how many frontier loops/arcs are consumed.
2. **Production data:** how many new frontier loops/arcs are created.
3. **Residual arc map:** the distinguished local arc $\rho(v)$ not already encoded by yarn-graph edges.
4. **Crossing list:** ordered tuples $(o; u_1, u_2, \varepsilon)$.
5. **Frontier replacement rule:** where the new frontier arcs sit in left-to-right order.
6. **Boundary effect:** whether the stitch changes the cast-on or working-end status.
7. **Matrix template:** the sparse local block B_σ added to the coloring system.

This is the exact amount of information needed for a compositional theorem and for implementation.

7A. Two Ways To Keep Track of Arcs

There are two viable designs. Both can be made rigorous, but they optimize for different things.

7A.1 Option 1: attach arcs to the yarn graph

In this design, the yarn graph remains the primary object, and diagram arcs are derived objects attached to yarn-graph incidences.

Data model.

1. Each yarn-graph edge e carries one or more arc labels $a_{e,1}, \dots, a_{e,r(e)}$.
2. Each stitch vertex v carries at most one residual local arc $\rho(v)$ not represented by a pre-existing yarn-graph edge.
3. Each stitch vertex stores a crossing list whose entries reference incident edge-arcs and possibly $\rho(v)$.

Why this matches your current intuition.

1. Most of the yarn path is already present in the yarn graph.
2. For generic stitches, only one arc appears to be missing, so one can attach that missing arc to the stitch vertex.
3. Crossing extraction becomes a local annotation pass on an already existing graph object.

Advantages.

1. Minimal duplication of data.
2. Easy to prove consistency with the existing implementation.
3. Good for algorithms that alternate between geometric layout and knot extraction.

Disadvantages.

1. Arcs are not first-class objects, so matrix extraction requires an extra normalization step.
2. Some crossings will be expressed in mixed coordinates: partly by yarn-graph edges and partly by stitch-local residual arcs.
3. Complicated stitches such as cables, wrap-turns, and bind-off may require edge subdivision conventions.

Recommended schema.

$$e \mapsto (\text{tail}, \text{head}, \text{orientation}, \text{arc labels}) \quad v \mapsto (\rho(v), C(v), \text{frontier action})$$

Then each crossing is stored as

$$(o; u_1, u_2, \varepsilon, \text{stitch id}).$$

7A.2 Option 2: list arcs only at the stitch level

In this design, the crossing presentation is independent of yarn-graph edges. Each stitch explicitly contributes all of its local arcs, and gluing occurs directly between stitch interfaces.

Data model.

1. Every stitch σ has a local arc set

$$A_\sigma = A_\sigma^{\text{in}} \sqcup A_\sigma^{\text{out}} \sqcup A_\sigma^{\text{int}}.$$

2. The global diagram is formed by gluing outgoing interface arcs of one stitch to incoming interface arcs of adjacent stitches according to the stitch program and row/needle order.
3. The yarn graph may still be derived afterward as a quotient or shadow of the arc system.

Advantages.

1. Arcs are first-class objects from the start.
2. Crossing matrices are immediate to assemble.
3. Theorems about knot diagrams are cleaner because the primary object is already diagrammatic.

Disadvantages.

1. More duplication, since the same yarn path information is stored again in a second structure.
2. Harder to maintain consistency with the existing yarn graph unless a formal equivalence theorem is proved.
3. Implementation is heavier because every stitch must declare all of its local arcs explicitly.

Recommended schema.

$$v \mapsto (A_v^{\text{in}}, A_v^{\text{out}}, A_v^{\text{int}}, C(v), \text{gluing rule}, \text{frontier action})$$

and the entire finished object is obtained by iterated gluing of stitch interfaces.

7A.3 Comparison

Criterion	Arcs attached to yarn graph	Arcs listed only with stitches
Closeness to current code	Very high	Moderate
Ease of invariant extraction	Moderate	High
Data duplication	Low	Higher
Proof burden for representation	Lower initially	Cleaner once formalized
Handling generic stitches	Very good	Good
Handling cables/bind-off/wrap-turn	Requires careful edge subdivision	Natural but more verbose

7A.4 Recommendation

For this project, the best immediate choice is a hybrid that is mathematically phrased as Option 1:

1. keep the yarn graph as the master combinatorial object,
2. attach arc labels to yarn-graph edges,
3. attach one residual arc and a crossing list to each stitch vertex,
4. define a normalization pass that converts this annotated yarn graph into a pure arc-and-crossing diagram when needed.

This is the least disruptive representation and appears to match your observation that a generic stitch has only one arc not already accounted for by yarn-graph edges. If later the mathematics wants a cleaner primary object, Option 2 can be introduced as a derived equivalent presentation.

8. Immediate Mathematical Tasks

1. Fix one orientation convention for the diagram plane and assign signs for knit and purl crossings.
2. Draw the minimal local knot-diagram gadget for: first cast-on, generic cast-on, wrap-turn, knit, purl, bind-off, and final bind-off pull-through.
3. For each gadget, enumerate interface arcs, internal arcs, and crossings.
4. Define the frontier update rule for each gadget.
5. Write the corresponding local coloring block B_σ for each gadget.
6. Prove local preservation of single-strandness for each primitive.
7. Prove that composing gadgets yields the same closed diagram as following the yarn graph.
8. Only after the coloring theorem is finished, extend to an Alexander matrix statement.

9. Best Next Theorem Statements

If the goal is an efficient paper and implementation path, the best theorem sequence is:

1. **Representation theorem.** Every valid stitch program induces a unique yarn crossing presentation by local gadget gluing.
2. **Single-component theorem.** The closure of every valid finished program is one closed component.
3. **Local coloring update theorem.** Each stitch acts by a bounded update on the frontier-reduced coloring matrix.
4. **Determinant theorem.** The determinant of the finished closure is computable in polynomial time from those local updates.
5. **Extension theorem.** The same presentation yields an Alexander matrix; Jones is computable afterward from the completed diagram.

Bottom Line

The right abstraction is not “just the yarn graph” and not “just a crossing list.” It is a stitch-local crossing semantics layered on top of the yarn graph, with one residual arc and a constant-size crossing gadget per stitch. The best proof engine is then a frontier-reduced Fox coloring matrix updated by constant-size block operations at each stitch. That gives a realistic path to a compositional theorem, an implementation, and a determinant computation.